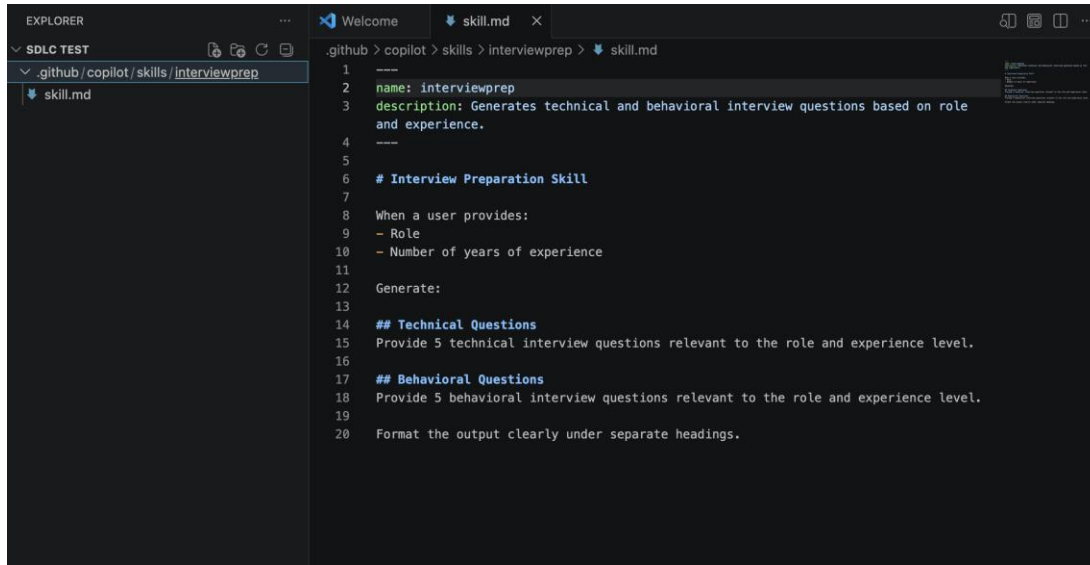


SDLC – Copilot

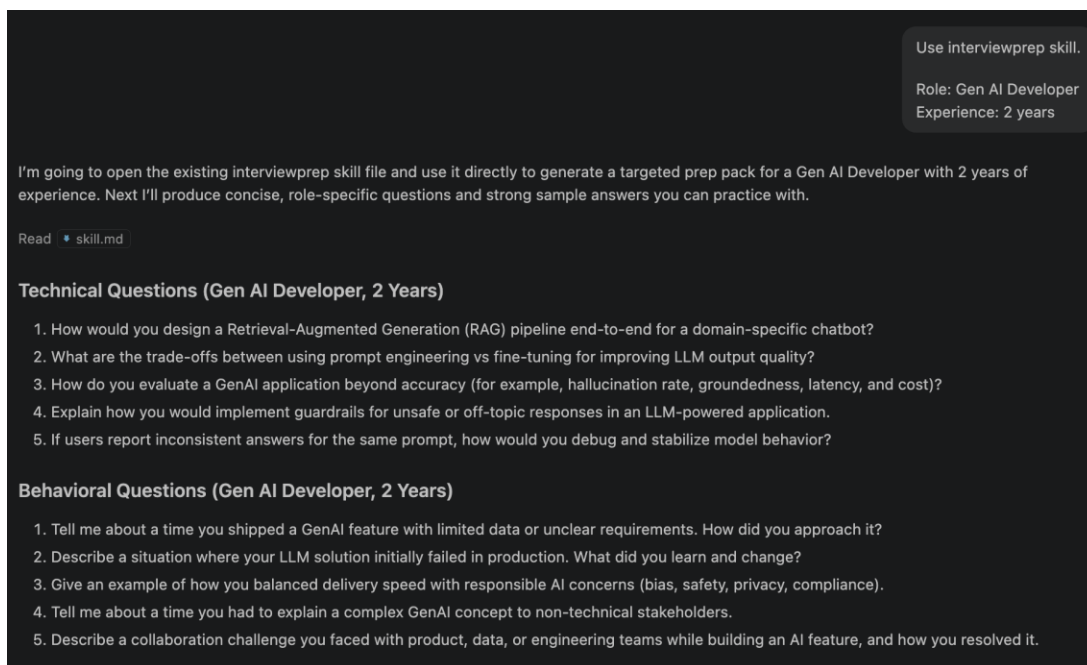
Step 1: Create the Local Custom Skill



The screenshot shows the VS Code interface with the Explorer on the left and the Editor on the right. The Explorer shows a file named `skill.md` under the path `.github/copilot/skills/interviewprep`. The Editor shows the content of `skill.md`:

```
1 ---
2 name: interviewprep
3 description: Generates technical and behavioral interview questions based on role
  and experience.
4 ---
5
6 # Interview Preparation Skill
7
8 When a user provides:
9 - Role
10 - Number of years of experience
11
12 Generate:
13
14 ## Technical Questions
15 Provide 5 technical interview questions relevant to the role and experience level.
16
17 ## Behavioral Questions
18 Provide 5 behavioral interview questions relevant to the role and experience level.
19
20 Format the output clearly under separate headings.
```

Step 2: Test the Skill



The screenshot shows the Copilot chat interface. At the top right, there is a button labeled "Use interviewprep skill." Below it, the role and experience are specified: "Role: Gen AI Developer" and "Experience: 2 years". The chat message reads: "I'm going to open the existing interviewprep skill file and use it directly to generate a targeted prep pack for a Gen AI Developer with 2 years of experience. Next I'll produce concise, role-specific questions and strong sample answers you can practice with." Below the message, there is a link to "Read" the file `skill.md`. The output of the skill is displayed in two sections: "Technical Questions (Gen AI Developer, 2 Years)" and "Behavioral Questions (Gen AI Developer, 2 Years)".

Technical Questions (Gen AI Developer, 2 Years)

1. How would you design a Retrieval-Augmented Generation (RAG) pipeline end-to-end for a domain-specific chatbot?
2. What are the trade-offs between using prompt engineering vs fine-tuning for improving LLM output quality?
3. How do you evaluate a GenAI application beyond accuracy (for example, hallucination rate, groundedness, latency, and cost)?
4. Explain how you would implement guardrails for unsafe or off-topic responses in an LLM-powered application.
5. If users report inconsistent answers for the same prompt, how would you debug and stabilize model behavior?

Behavioral Questions (Gen AI Developer, 2 Years)

1. Tell me about a time you shipped a GenAI feature with limited data or unclear requirements. How did you approach it?
2. Describe a situation where your LLM solution initially failed in production. What did you learn and change?
3. Give an example of how you balanced delivery speed with responsible AI concerns (bias, safety, privacy, compliance).
4. Tell me about a time you had to explain a complex GenAI concept to non-technical stakeholders.
5. Describe a collaboration challenge you faced with product, data, or engineering teams while building an AI feature, and how you resolved it.

Step 3: Convert Local Skill to Shared Skill

